Trust, Not Just Relevance: Verifier-Anchored Tool Selection Under Partial Feedback and Tool Drift

Author Name
Affiliation
email@example.com

Abstract

LLM agents choose tools by how well a tool's description matches the request. In open tool ecosystems such as the Model Context Protocol (MCP), that is unsafe: descriptions can be inflated or poisoned, tools report their own success, and tools change without notice. Learning from outcomes helps only if the outcomes can be trusted, and in practice only some calls can be checked. We study tool selection in this setting and describe RRL, a training-free selection layer that combines relevance with Beta reputation, learns how far to trust each feedback source (per tool) against the calls that were verified, keeps per-context memory that only verified outcomes can lock in, and resets reputations when a statistical test detects a change. We evaluate it on six benchmarks, including tool selection over live MCP servers with a poisoned server, tool selection on real tool descriptions from ToolRet, and executable web-scraping extractors. With a poisoned server present, a cross-encoder reranker and an LLM choosing from the tool list pick it almost every time and solve at most 8.2% of requests. When only 20% of calls are verified, a MemRL-style learner trusts the poisoned tool's own success reports and solves 3.9%, a robust per-context memory solves 24.8%, and RRL solves 52.4% (+27.6 points over the memory, 95% CI [+25.7, +29.5], better on 30 of 30 seeds). The gains are specific to that regime: when every call is verified and contexts recur, a simple per-context memory beats RRL by 8–17 points, and under abrupt API changes with trustworthy feedback a fast-forgetting learner adapts faster. We release the code, recorded outcomes and replay harness.

1 Introduction

An agent with access to many tools must decide which one to call. Today this choice is made from text: a retriever or the LLM itself compares the request with each tool's name and description [12, 22]. This assumes descriptions are honest and tools behave as described. In open registries neither holds. MCP servers can carry poisoned descriptions that both attract the agent and inject instructions [11, 24], tools are renamed and change their arguments between versions [4, 16], and agents readily adopt the outputs of unreliable tools [28]. Defences that rewrite or isolate descriptions [21, 30] address injection but not the question of which tool actually works.

The natural alternative is to learn from outcomes. Runtime memory methods such as MemRL [32] and Memento [33] learn which stored items lead to success, and RATED [5] learns tool reliability from execution feedback with a contextual bandit. These methods take feedback at face value. But the cheapest feedback is often a tool's own report that it succeeded, or a schema check that a well-formed guess passes; expensive verification (running tests, a human check, comparing with ground truth) is affordable only for a fraction of calls [18, 31]. A tool that over-reports success then looks like the best tool, and a learner that trusts the reports locks it in.

Our starting point was web scraping, where the same problem appears in a simple form: which extractor works for which site, when extractors report success on pages they parsed wrongly and sites redesign. We study the general problem: *online tool selection when tools may mislead, only some calls are verified, and tools change*. We make three contributions.

1. **Verifier-anchored feedback trust in tool selection.** A two-coin [7] model of each feedback source's sensitivity and specificity, estimated against verified outcomes globally and per tool, turns unverified signals into a posterior success probability; a source that cannot tell success from failure gets zero weight.

2. **A training-free selection layer** that combines relevance, Beta reputation [14], Thompson exploration [23], per-context counters shrunk toward global ones, verified-success memory and z-test change detection.

3. **A reproducible evaluation** on six benchmarks, including live MCP servers with a poisoned server and an LLM choosing from the tool list, with 30 paired seeds, ablations, and an explicit map of where simpler methods win.

Relation to the closest work. Two recent papers cover our two main ideas separately. RATED [5] learns which tools work from execution feedback but assumes the feedback is truthful; ESA [10] learns per-source, per-context trust from sparse ground-truth audits, with regret bounds, but has no tool or LLM application. Our contribution is not a new estimator: it is bringing audit-anchored source trust into tool selection for agents whose tools mislead through their descriptions and their own success reports, adding per-context memory that only verified outcomes can lock in, and an evaluation that shows when this matters and when it does not. We have not yet compared against the official implementations of RATED, ESA or MemRL (Section 7).

2 Problem setting

At each step t the agent receives a request q_t with a context c_t (for example the website or the API being used) and a candidate pool $\mathcal{D}_t$ of tools with relevance scores $\mathrm{sim}(q_t, d) \in [0, 1]$ from a retriever or reranker. It calls one tool d_t and observes feedback signals $s_t = (s_t^k)_k$ from several sources k: the tool's own status report, a schema validator or judge, and user behaviour. With probability ρ (the *verification rate*) it also observes the true outcome $y_t \in \{0, 1\}$ from a verifier. The goal is to maximise the true success rate $\frac{1}{T}\sum_t y_t$, including on contexts that appear only late.

Three properties make the problem hard. (i) *Misleading relevance*: the most relevant description may belong to a poisoned or inflated tool. (ii) *Misleading feedback*: a source may report success regardless of the outcome, and how often it does so can differ by tool. (iii) *Non-stationarity*: tools are updated or removed and contexts change (a site redesign), so past outcomes go stale. We call $\rho = 1$ the *verified* regime and $\rho = 0.2$ the *spot-check* regime.

3 Method

RRL keeps, for each tool d, short-term Beta counters (α_d, β_d), slower long-term counters (A_d, B_d), and per-context counters $(\alpha_{d,c}, \beta_{d,c})$.

Scoring. Per-context counters are shrunk toward the global ones by the evidence they hold, $n_{d,c} = \alpha_{d,c} + \beta_{d,c} - 2$:

$$\lambda = \frac{n_{d,c}}{n_{d,c} + \kappa}, \qquad \alpha = (1-\lambda)\,\alpha_d + \lambda\,\alpha_{d,c}, \qquad \beta = (1-\lambda)\,\beta_d + \lambda\,\beta_{d,c}, \tag{1}$$

with $\kappa = 4$, and likewise for (A, B). A new context therefore starts from the tool's global reputation and its own outcomes take over as they accumulate. The score is

$$\text{score}(d) = w_s\,\text{sim}(q,d) + w_c\,\frac{\alpha}{\alpha+\beta} + w_p\,\frac{A}{A+B} + w_e\,\text{sim}(q,d)\,(\theta_d + b_d), \tag{2}$$

where $\theta_d \sim \text{Beta}(\alpha, \beta)$ is a Thompson sample and $b_d = \max(0.05, (\alpha+\beta)^{-1/2})$ an uncertainty bonus; scaling exploration by relevance avoids trying tools that are irrelevant to the request.

Verified-success memory. If the verifier has confirmed that tool d worked in context c and no verified failure has followed, d is ranked first for c and exploration is skipped. Only the verifier can set or clear this flag, so a tool cannot lock itself in through its own reports.

Feedback trust. When the verifier ran, the outcome used for learning is y_t. When it did not, RRL combines the other signals using what it has learned about each source. For source k and tool d it keeps counts, over verified calls, of how often k said "success" when the call succeeded (sensitivity $\text{se}_{k,d}$) and "failure" when it failed (specificity $\text{sp}_{k,d}$), shrunk toward the source's global rates with prior strength 4. Each signal contributes a log-likelihood ratio

$$\ell_k = \begin{cases} \log \dfrac{\text{se}_{k,d}}{1-\text{sp}_{k,d}} & \text{if } k \text{ reports success,} \\[2ex] \log \dfrac{1-\text{se}_{k,d}}{\text{sp}_{k,d}} & \text{if } k \text{ reports failure,} \end{cases} \tag{3}$$

clipped to $[-4, 4]$ and set to 0 when $\text{se}_{k,d} + \text{sp}_{k,d} \le 1$, that is, when the source is no better than chance for this tool. The learning target is

$$\hat{y}_t = \sigma\Big(\text{logit}\,\pi + \textstyle\sum_k \ell_k\Big), \tag{4}$$

where π is the verified base rate; if no source is informative the step is not used for learning. This is the two-coin Dawid–Skene model [7, 26] anchored to a verifier rather than estimated by EM, close in spirit to trust bootstrapping from a small trusted set [2] and to audit-based trust in ESA [10]. Estimating the rates per tool matters: a poisoned tool's self-report is uninformative for that tool even if the same kind of report is informative for the others.

Updates. After each call the chosen tool's counters are updated with $\hat{y}$: $\alpha \mathrel{+}= \omega\hat{y}$, $\beta \mathrel{+}= \omega(1-\hat{y})$, with weight $\omega = 2|\hat{y} - \frac{1}{2}|$ so that uncertain outcomes move the counters little; the long-term counters receive a quarter of the update.

Change detection. For each tool (and each tool–context pair) RRL compares the success rates $\hat{p}_1, \hat{p}_2$ of the last two windows of $w = 8$ outcomes with a two-proportion z-test at $\alpha = 0.01$, $z = |\hat{p}_2 - \hat{p}_1| / \sqrt{2\bar{p}(1-\bar{p})/w}$. On detection the short-term counters are rebuilt from the most recent window only. This replaces a fixed threshold on $|\hat{p}_2 - \hat{p}_1|$, which produced many false resets (Section 5.1); it is a simple instance of adaptive windowing [1] and related to sliding-window and discounted bandits [9, 19].

4 Benchmarks

All benchmarks replay offline from recorded outcomes: for every request and every tool in its pool we record whether the tool works, so any policy can be evaluated on the same data [15]. Seeds vary the request order, feedback noise, flaky calls and exploration; we run 30 seeds and pair every comparison over the same seeds (95% CIs from a paired t-test; Wilcoxon tests agree). A held-out set of contexts appears only in a final block, to measure transfer.

Gate M: live MCP servers. Four local MCP servers over stdio expose eight extractor tools that fetch pages from a local shop site and return title, price and SKU: CSS extractors, a structured-data server (JSON-LD, microdata, Open Graph), a heuristic scraper on a flaky server, and a "universal AI extractor" whose description is inflated and carries an MCPTox-style [24] instruction to call `send_report` with the page content. The canary tool only appends a line to a local file, so following the injection can be counted. 6 of 30 shops are held out. In the *update* scenarios half the shops redesign and the structured-data server is upgraded mid-run, renaming `extract_jsonld(url)` to `jsonld_extract(page_url)` as in MCPEvol-Bench's mutations [16]. Outcomes were recorded from 2,280 live MCP calls and replayed with 20% timeouts on the flaky server and 2% transient failures on every tool. We also run every policy with real MCP calls on every step. The *LLM picks* baseline gives Gemini 2.5 Flash the live tool list and lets it plan its calls, as agents do today.

Gate W: web-scraping extractors. 30 generated shops (6 held out) and the same eight extractors running real parsing code, without the MCP layer. The universal extractor returns well-formed guesses, so both its self-report and the schema validator say success.

Gate T: real tool descriptions. Requests and gold tools from ToolRet [22] (ToolBench: 431 requests whose top-8 pool contains a working tool; ToolLens: 200). On top we add v2 replacements for 30% of gold tools (drift), poisoned twins for 10% whose descriptions copy example requests (the reranker then picks a poisoned tool first for 39% and 56% of requests), and flakiness for 20% of tools. Success means the chosen tool is a gold tool that is deployed and does not flake; ToolRet's labels stand in for execution.

Gate G: private library. Gemini 2.5 Flash writes code against a generated library whose documentation is the only way to solve the task; poisoned guides fool the reranker on 79 of 80 poisoned questions; *drift* changes the library's API halfway.

Gate F: MBPP. A leak-free retrieval benchmark on MBPP with 4,128 recorded Gemini 2.5 Flash completions, used as a control.

Feedback scenarios. *Verified*: every call is verified. *Spot check*: 20% of calls are verified and the rest carry only the tool's report and the schema check. *Sycophantic* and *fooled* (Gates T, G): the unverified judge says success too often, in general or specifically for poisoned items.

Baselines. *Static reranker*: cross-encoder top-1. *LLM picks* (Gate M). *Per-context memory*: for each context, walk the reranker's order until a tool works, then keep it. Its *robust* variant keeps a working tool through an isolated failure (two failures in a row, three if unverified, before moving on). *MemRL-style*: our reimplementation of MemRL's ranking rule [32], $(1-\lambda)\,z(\text{sim}) + \lambda\,z(Q)$ with $Q \leftarrow Q + 0.3(r - Q)$. The learning baselines use the verifier when it ran and take the other

Overall / held-out success (%)	Verified	Spot check	Update	Update + spot
Static reranker	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0
LLM picks from tool list	1.0 / 0.0	1.0 / 0.0	8.2 / 0.0	8.2 / 0.0
Per-context memory	70.9 / 65.0	51.8 / 20.3	62.9 / 61.3	**46.5** / 17.9
Per-context memory, robust	**75.0** / 65.3	24.8 / 11.1	**65.7** / 61.4	21.9 / 10.1
MemRL-style	35.8 / 45.8	3.9 / 1.9	33.8 / 32.2	3.2 / 1.2
RRL (global)	45.9 / 57.2	42.1 / 60.3	42.5 / 51.2	37.5 / 44.4
RRL full	66.6 / 79.3	**52.4** / 72.5	56.5 / 66.5	43.4 / 52.8
Best possible	90.4 / 98.8	90.4 / 98.8	90.4 / 98.8	90.4 / 98.8

Table 1: Gate M: tool selection over live MCP servers with a poisoned server (replay of recorded MCP responses, 30 seeds). Best non-oracle result per column in bold.

feedback at face value. *Best possible*: an oracle that always picks a working tool if one exists. *RRL (global)* is RRL without per-context counters or verified memory; *RRL full* has both. The benchmark weights are $(w_s, w_c, w_p, w_e) = (0.20, 0.40, 0.10, 0.30)$; the library default uses $w_e = 0.10$, which we report as an ablation.

5 Results

Description-based selection fails with a poisoned server. With the poisoned server in the registry, the static reranker picks it on every step and solves none of the requests (Table 1). Gemini choosing from the tool list does little better: it picks the poisoned tool on 99.0% of steps, solves 1.0–8.2% of requests, and plans the injected `send_report` call on 4.2–6.4% of steps. Every learning method recovers: with every call verified RRL full solves 66.6% and picks the poisoned tool on 1.4% of steps.

With spot checks, RRL is the learner that is not fooled. When only 20% of calls are verified, the poisoned tool's own success reports dominate the feedback. MemRL-style trusts them, picks the poisoned tool on 90.1% of steps and solves 3.9%. The robust per-context memory, built to survive isolated failures, holds on to the poisoned tool on the strength of the same reports (68.8% poisoned picks, 24.8% solved). RRL full learns that the self-report is uninformative for that tool, picks it on 13.0% of steps, and solves 52.4%: +27.6 points over the robust memory (95% CI [+25.7, +29.5], better on 30 of 30 seeds). Gate W shows the same pattern without the MCP layer (+26.1 points; Table 2). RRL is also far better on held-out contexts, because a new context starts from each tool's global reputation rather than from the reranker.

When every call is verified, a per-context memory wins. In the verified regime there is nothing to be fooled by, and the robust memory beats RRL full by 8.4 points on Gate M (95% CI [6.6, 10.3]), by 11–16 on Gate W, and by 15–17 on the clean registry (Table 3). Without the poisoned server it also wins under spot checks, by 19–21 points: when no tool over-reports success, the unverified reports are honest and trusting them is right. Per-context memory is the right tool when outcomes are trustworthy and contexts recur; RRL buys robustness to misleading feedback and transfer to new contexts at the cost of exploration.

Live runs agree with the replay. Table 4 compares the replay with runs that make real MCP calls on every step. RRL full is within about 2 points of its replayed result. Per-context memory

Overall / held-out success (%)	Verified	Spot check	Redesign	Redesign + spot
Static reranker	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0
Per-context memory	67.0 / 61.4	50.4 / 19.6	64.2 / 58.5	**47.9** / 17.5
Per-context memory, robust	**73.9** / 61.9	27.0 / 11.0	**70.4** / 58.9	25.2 / 10.3
MemRL-style	35.9 / 43.3	4.4 / 3.6	34.0 / 32.1	4.3 / 3.5
RRL (global)	51.4 / 68.9	46.4 / 72.9	45.5 / 53.5	39.4 / 54.0
RRL full	62.5 / 85.7	**53.1** / 79.6	54.1 / 66.5	45.4 / 61.9
Best possible	87.9 / 96.4	87.9 / 96.4	87.9 / 96.4	87.9 / 96.4

Table 2: Gate W: web-scraping extractor selection with executable extractors.

Overall / held-out success (%)	Verified	Spot check	Update	Update + spot
Static reranker	46.0 / 65.7	46.0 / 65.7	37.4 / 49.3	37.4 / 49.3
LLM picks from tool list	47.7 / 61.8	47.7 / 61.8	49.7 / 53.8	49.7 / 53.8
Per-context memory	77.5 / 89.4	77.0 / 89.4	70.4 / 85.4	69.2 / 85.4
Per-context memory, robust	**81.8** / 90.3	**81.6** / 90.3	**72.5** / 86.0	**71.3** / 86.0
MemRL-style	34.9 / 45.6	34.2 / 40.8	34.1 / 31.7	31.7 / 26.9
RRL (global)	48.0 / 61.9	47.7 / 58.3	44.1 / 51.9	44.1 / 50.8
RRL full	66.3 / 75.7	61.0 / 68.8	56.0 / 65.0	52.7 / 63.6
Best possible	90.4 / 98.8	90.4 / 98.8	90.4 / 98.8	90.4 / 98.8

Table 3: Gate M without the poisoned server (clean registry).

does about 6 points better live, because the replay adds 2% transient failures on every tool, the local servers had none outside the flaky one, and that baseline abandons a working tool after a single failure.

On real tool descriptions RRL matches MemRL-style. On ToolRet requests (Tables 5 and 6), RRL and MemRL-style are within a few points of each other: MemRL-style is ahead by 3–4 points under the sycophantic judge, RRL full is ahead by 2 points on ToolLens with the fooled judge, and with the library's exploration weight RRL matches or slightly exceeds MemRL-style in most scenarios. Both beat per-context memory by about 5–16 points because requests rarely repeat, and both remain far from the best possible pick.

Under abrupt change, a fast-forgetting learner adapts faster. In Gate G (Table 7), when the library's API changes halfway and feedback is trustworthy, MemRL-style's exponential moving average forgets the old documents faster than RRL's change detector, which waits for statistical evidence. In the drift scenario RRL (global) trails it by 9 points and RRL full by 21.2: per-context counters and verified memory keep the old documents for questions that recur after the change.

A control: when the model knows the answer, retrieval does not matter. On MBPP (Gate F), the reranker solves 57.1% and no retrieval 57.2%; the best possible pick reaches only 61.3%. A benchmark where the model already knows the answer cannot show the value of learning which items help, which is why we built Gates G, T, W and M.

5.1 Ablations

Table 8 removes one component at a time on Gate M.

Success (%)	Verified		Update	
	Replay	Live	Replay	Live
Static reranker	0.0	0.0	0.0	0.0
LLM picks from tool list	1.0	1.0	8.2	8.3
Per-context memory	70.9	77.2	62.9	69.4
Per-context memory, robust	75.0	77.2	65.7	67.9
MemRL-style	35.8	37.7	33.8	35.0
RRL (global)	45.9	47.4	42.5	44.3
RRL full	66.6	66.2	56.5	58.6

Table 4: Gate M replayed from recorded MCP responses (30 seeds) versus run with real MCP calls on every step (3 seeds).

Overall / held-out success (%)	Verified	Sycophantic	Fooled	Drift	Drift + fooled
Static reranker	31.4 / 33.0	31.4 / 33.0	31.4 / 33.0	30.5 / 30.3	30.5 / 30.3
Per-context memory	46.0 / 33.0	41.1 / 33.0	39.9 / 33.0	41.0 / 30.3	35.2 / 30.3
Per-context memory, robust	46.4 / 33.0	39.8 / 33.0	39.7 / 33.0	40.4 / 30.3	34.1 / 30.3
MemRL-style	**55.5** / 45.3	**50.0** / 42.7	**45.5** / 39.2	**51.0** / 42.3	41.9 / 38.2
RRL (global)	52.5 / 50.1	45.2 / 46.6	44.2 / 43.3	48.1 / 48.7	41.4 / 44.0
RRL full	55.3 / 48.4	45.9 / 45.6	45.0 / 42.8	49.5 / 48.5	**42.0** / 42.6
Best possible	89.9 / 89.4	89.9 / 89.4	89.9 / 89.4	88.6 / 87.1	88.6 / 87.1

Table 5: Gate T (ToolBench requests from ToolRet): tool selection on real tool descriptions.

- **Per-tool trust matters most under spot checks.** Using only each source's global sensitivity and specificity costs 7.1 points on Gate M and 8.1 on Gate W: the poisoned tool's report is informative on average but not for that tool.
- **Per-context counters and verified memory** add the largest gains where contexts recur; verified memory alone adds +8.8 points with every call verified. Neither helps on Gate T, where requests rarely repeat, and both hurt under drift on Gate G (above).
- **Change detection.** The z-test detector cuts resets in stable conditions from 23.0 to 1.9 per run on Gate W, compared with the fixed-threshold rule, but reacts later to a real change (Gate G).
- **Exploration.** A confidence gate that stops exploring once the leader is significantly best had no effect: while any tool is untried, the leader cannot be shown best at 95% confidence. The library default $w_e = 0.10$ is better than the benchmark's 0.30 on Gates T and G; we report both rather than re-tune after seeing the results.

6 Related work

Tool retrieval and selection. ToolRet [22] shows that retrievers are weak at finding the right tool; AutoTool [12] reduces selection cost; LQM-ContextRoute [6] routes among functionally equivalent tools by latency and quality. RATED [5] is the closest: it learns tool reliability from execution dynamics with a contextual bandit, assuming truthful feedback.

Overall / held-out success (%)	Verified	Sycophantic	Fooled	Drift	Drift + fooled
Static reranker	29.1 / 32.2	29.1 / 32.2	29.1 / 32.2	26.5 / 25.2	26.5 / 25.2
Per-context memory	42.9 / 32.2	38.2 / 32.2	34.7 / 32.2	36.2 / 25.2	29.7 / 25.2
Per-context memory, robust	43.5 / 32.2	37.1 / 32.2	34.3 / 32.2	35.7 / 25.2	28.6 / 25.2
MemRL-style	55.4 / 57.9	**51.7** / 56.3	43.9 / 50.3	**52.4** / 55.4	40.6 / 42.0
RRL (global)	56.1 / 55.6	48.1 / 52.1	46.2 / 50.3	49.9 / 52.0	41.6 / 42.3
RRL full	**58.1** / 55.8	48.7 / 51.1	**46.3** / 49.8	50.4 / 51.2	**41.8** / 45.0
Best possible	85.1 / 81.1	85.1 / 81.1	85.1 / 81.1	84.7 / 83.2	84.7 / 83.2

Table 6: Gate T (ToolLens requests from ToolRet).

Overall / held-out success (%)	Verified	Fooled	Drift	Drift + syco.	Drift + fooled
Static reranker	50.8 / 50.0	50.8 / 50.0	23.5 / 0.0	23.5 / 0.0	23.5 / 0.0
Per-context memory	83.7 / 50.0	64.8 / 50.0	51.3 / 0.0	39.1 / 0.0	40.1 / 0.0
Per-context memory, robust	83.7 / 50.0	59.4 / 50.0	34.8 / 0.0	29.8 / 0.0	26.3 / 0.0
MemRL-style	96.0 / 100.0	**89.7** / 99.4	**80.9** / 95.5	**73.7** / 93.8	**74.8** / 93.9
RRL (global)	96.5 / 100.0	88.0 / 98.8	71.7 / 96.7	52.4 / 65.2	56.7 / 75.7
RRL full	**96.5** / 100.0	88.1 / 99.4	59.8 / 74.7	46.9 / 53.1	50.5 / 70.1
Best possible	100.0 / 100.0	100.0 / 100.0	100.0 / 100.0	100.0 / 100.0	100.0 / 100.0

Table 7: Gate G (hard): code generation against a generated library with poisoned guides and a mid-stream API change.

Tool poisoning and evolution. MCPTox [24] and MCP threat models [11] show that poisoned descriptions succeed against real agents; Tool-Guard [21] and TRUSTDESC [30] defend by isolated planning and trusted description generation. MCPEvol-Bench [16] and ToolEVO [4] study tools that change. Agents over-rely on unreliable tools [28]. Our approach is complementary to these defences: it does not inspect descriptions, it learns from outcomes which tools to stop using.

Learning from feedback in agents and retrieval. MemRL [32], Memento [33] and Evo-Memory [25] learn the utility of stored items at run time; RoMeRL [29] discusses how memory rewards can trap such learners. Rerankers trained from LLM feedback [27] and utility-driven reranking [3] also rank by outcome rather than similarity, but offline. Feedback loops can degenerate [13]. Verified semantic caching [20] and prediction-powered ranking evaluation [8] rely on verification or a small labelled sample to make cheaper signals trustworthy, as we do.

Trust and robust bandits. Beta reputation [14], the Dawid–Skene model [7, 26], trust bootstrapping [2] and contextual trust learning from audits [10] underlie our feedback model. Corruption-robust bandits [17] bound the damage of adversarial feedback without modelling its source; non-stationary bandits [9, 19] and adaptive windowing [1] handle change. Pan et al. [18] show that self-reports of LLM operators are not verification, and Zavattari et al. [31] study how to allocate a limited audit budget across agents.

7 Limitations

- **Baselines.** MemRL-style is a reimplementation of MemRL's ranking rule, not its official code, and we have not yet compared against RATED or an ESA-style aggregator, the two

Overall / held-out success (%)	Verified	Spot check	Update	Update + spot
Relevance + reputation only	48.3 / 65.0	44.3 / 63.5	45.3 / 60.8	38.9 / 52.2
Global-only source trust	45.9 / 57.2	35.0 / 53.3	42.5 / 51.2	28.5 / 27.1
No change detection	48.3 / 65.0	43.7 / 64.4	45.3 / 60.8	38.8 / 51.8
Fixed-threshold detector	42.6 / 57.2	38.2 / 55.4	39.3 / 39.7	33.6 / 32.5
Exploration off	44.6 / 55.3	42.2 / 52.8	40.7 / 45.1	38.0 / 36.2
Exploration 0.10	44.6 / 51.0	42.9 / 53.5	41.6 / 46.5	39.8 / 46.9
Confidence-gated exploration	45.9 / 57.2	42.1 / 60.3	42.5 / 51.2	37.5 / 44.4
RRL (global)	45.9 / 57.2	42.1 / 60.3	42.5 / 51.2	37.5 / 44.4
RRL + per-context	57.8 / 67.6	48.8 / 69.4	49.8 / 60.4	41.1 / 55.7
RRL full	**66.6** / 79.3	**52.4** / 72.5	**56.5** / 66.5	**43.4** / 52.8

Table 8: Ablations on Gate M (overall / held-out success, %). Each row changes one component of RRL (global); the last two rows add per-context counters and verified memory.

closest methods.

- **Environments.** Gate M uses real MCP servers over stdio but on a local, generated website with a synthetic poisoned server; no third-party MCP servers or live websites were used. We have not evaluated on unmodified MCPTox or MCPEvol-Bench tasks. Gate T judges success by ToolRet labels, not execution.
- **Synthetic stress.** Poisoning, flakiness, updates, redesigns and the feedback models are added on top of the data; their rates are our choices.
- **Feedback trust.** We assume verified calls are sampled at random. Biased audits, correlated sources, and deception that adapts to the audit policy are untested and could break the trust estimates.
- **Regime.** RRL loses to a per-context memory when every call is verified and to a fast-forgetting learner under abrupt change with trustworthy feedback.

8 Conclusion

Choosing tools by their descriptions fails as soon as a description lies, and learning from outcomes fails as soon as the outcomes lie. Anchoring feedback trust to the few calls that can be verified, per tool, lets a simple training-free selection layer avoid a poisoned tool that a reranker, an LLM and two learning baselines all adopt, and transfer what it learned to new contexts. Where every call can be checked, simpler memories are better. The open questions are how to choose which calls to verify, and how robust audit-anchored trust is to adversaries that know the audit policy.

Reproducibility. Code, recorded outcomes, per-seed results and the scripts that generate every table are available at `https://github.com/pras-ops/retrieval-reputation-layer`.

References

[1] Albert Bifet and Ricard Gavaldà. Learning from Time-Changing Data with Adaptive Windowing. In *Proceedings of the SIAM International Conference on Data Mining*, 2007.

[2] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. FLTrust: Byzantine-robust Federated Learning via Trust Bootstrapping, 2020. URL https://arxiv.org/abs/2012.13995.

[3] Manish Chandra, Debasis Ganguly, and Iadh Ounis. LURE-RAG: Lightweight Utility-driven Reranking for Efficient RAG, 2026. URL https://arxiv.org/abs/2601.19535.

[4] Guoxin Chen, Zhong Zhang, Xin Cong, Fangda Guo, Yesai Wu, Yankai Lin, Wenzheng Feng, and Yasheng Wang. Learning Evolving Tools for Large Language Models, 2024. URL https://arxiv.org/abs/2410.06617.

[5] Weiliang Chen and Song Liao. RATED: Reliability-Aware AI Agent Tool Retrieval via Execution Dynamics, 2026. URL https://arxiv.org/abs/2610.13889.

[6] Kexin Chu, Dawei Xiang, and Wei Zhang. Latency-Quality Routing for Functionally Equivalent Tools in LLM Agents, 2026. URL https://arxiv.org/abs/2605.14241.

[7] A. P. Dawid and A. M. Skene. Maximum Likelihood Estimation of Observer Error-Rates Using the EM Algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 28 (1):20–28, 1979.

[8] Abhishek Divekar. Statistically Reliable LLM-Based Ranking Evaluation via Prediction-Powered Inference, 2026. URL https://arxiv.org/abs/2606.05308.

[9] Aurélien Garivier and Eric Moulines. On Upper-Confidence Bound Policies for Non-Stationary Bandit Problems, 2008. URL https://arxiv.org/abs/0805.3415.

[10] Majid Ghasemi and Mark Crowley. Learning When to Trust in Contextual Social Bandits, 2026. URL https://arxiv.org/abs/2603.13356.

[11] Charoes Huang, Xin Huang, Ngoc Phu Tran, and Amin Milani Fard. Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning, 2026. URL https://arxiv.org/abs/2603.22489.

[12] Jingyi Jia and Qinbin Li. AutoTool: Efficient Tool Selection for Large Language Model Agents, 2025. URL https://arxiv.org/abs/2511.14650.

[13] Ray Jiang, Silvia Chiappa, Tor Lattimore, András György, and Pushmeet Kohli. Degenerate Feedback Loops in Recommender Systems, 2019. URL https://arxiv.org/abs/1902.10730.

[14] Audun Jøsang and Roslan Ismail. The Beta Reputation System. In *Proceedings of the 15th Bled Electronic Commerce Conference*, 2002.

[15] Lihong Li, Wei Chu, John Langford, and Xuanhui Wang. Unbiased Offline Evaluation of Contextual-bandit-based News Article Recommendation Algorithms, 2010. URL https://arxiv.org/abs/1003.5956.

[16] Huanxi Liu, Kun Hu, Jiaqi Liao, Qiang Wang, Pengfei Qian, YuanZhao Zhai, Dawei Feng, Bo Ding, and Huaimin Wang. MCPEvol-Bench: Benchmarking LLM Agent Performance Across Dynamic Evolutions of MCP Servers, 2026. URL https://arxiv.org/abs/2607.14642.

[17] Thodoris Lykouris, Vahab Mirrokni, and Renato Paes Leme. Stochastic bandits robust to adversarial corruptions, 2018. URL https://arxiv.org/abs/1803.09353.

[18] Enrong Pan, Ryan Zhou, and Ting Hu. Self-Reports Are Not Verification: Environment-Grounded Auditing of LLM Operators in Evolutionary Search, 2026. URL https://arxiv.org/abs/2609.00652.

[19] Han Qi, Yue Wang, and Li Zhu. Discounted Thompson Sampling for Non-Stationary Bandit Problems, 2023. URL https://arxiv.org/abs/2305.10718.

[20] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. vCache: Verified Semantic Prompt Caching, 2025. URL https://arxiv.org/abs/2502.03771.

[21] Shanghao Shi, Xiao Wang, Chaoyu Zhang, Hao Li, Wenjing Lou, Thomas Hou, Yevgeniy Vorobeychik, Chongjie Zhang, and Ning Zhang. Think Twice Before You Act: Protecting LLM Agents Against Tool Description Poisoning via Isolated Planning, 2026. URL https://arxiv.org/abs/2606.20922.

[22] Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models, 2025. URL https://arxiv.org/abs/2503.01763.

[23] William R. Thompson. On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples. *Biometrika*, 25(3/4):285–294, 1933.

[24] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers, 2025. URL https://arxiv.org/abs/2508.14925.

[25] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, et al. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory, 2025. URL https://arxiv.org/abs/2511.20857.

[26] Jacob Whitehill, Paul Ruvolo, Tingfan Wu, Jacob Bergsma, and Javier Movellan. Whose Vote Should Count More: Optimal Integration of Labels from Labelers of Unknown Expertise. In *Advances in Neural Information Processing Systems*, 2009.

[27] Yuhang Wu, Xiangqing Shen, Fanfan Wang, Cangqi Zhou, Zhen Wu, Xinyu Dai, and Rui Xia. Optimizing RAG Rerankers with LLM Feedback via Reinforcement Learning, 2026. URL https://arxiv.org/abs/2604.02091.

[28] Hoyeol Yang, Woojung Song, Taewon Kim, Jonghyun Song, Seoyeon Park, and Yohan Jo. Agents' Overreliance on Unreliable Tools, 2026. URL https://arxiv.org/abs/2609.05587.

[29] Yi Yang, Zhennan Chen, Yihong Zhuang, Tiehan Fan, Yinan Chen, Jian Li, Jian Yang, and Ying Tai. RoMeRL: Balancing Feedback Coverage and the Memory-Reward Trap in Self-Evolving Agent Memory via Reduced-Order Utility States, 2026. URL https://arxiv.org/abs/2608.02508.

[30] Hengkai Ye, Zhechang Zhang, Jinyuan Jia, and Hong Hu. TRUSTDESC: Preventing Tool Poisoning in LLM Applications via Trusted Description Generation, 2026. URL https://arxiv.org/abs/2604.07536.

[31] Cesare Zavattari, Alessandro Tommasi, and Giuseppe Prencipe. One Human, N Agents: Audit-Budget Allocation for LLM Agent Fleets under Miscalibrated, Correlated Confidence, 2026. URL https://arxiv.org/abs/2607.28317.

[32] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, et al. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory, 2026. URL https://arxiv.org/abs/2601.03192.

[33] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, et al. Memento: Fine-tuning LLM Agents without Fine-tuning LLMs, 2025. URL https://arxiv.org/abs/2508.16153.